Syntax and Semantics

Finite Automata & Regular Languages

Giorgio Bacci (grbacci@cs.aau.dk)

in this Lecture

- Strings and languages
- Finite automata (definition & examples)
- What is a computation
- Designing a finite automaton
- Regular operations

Strings and Languages

- An ***alphabet*** is a finite nonempty set

  Examples: $\Sigma_1 = \{0,1\}, \Sigma_2 = \{a, b, c, \ldots, x, y, z\}, \ldots$

- The members of an alphabet are called ***symbols***

- A ***string*** (or ***word***) ***over an alphabet*** is a finite sequence of symbols

  Examples: $0,\ 001,\ 0011, \ldots$ are all strings over Σ_1;

  $\text{word},\ \text{alphabet},\ \text{aaabbb}, \ldots$ are strings over Σ_2;

  Non-examples: $0 + 1,\ a \sim b, \ldots$ are ***not*** strings over Σ_1 or Σ_2

- The ***empty string***, written ε, is the string with no symbols

- A **language** over the alphabet Σ is a set of strings over Σ

  We denote by Σ^* the language of all strings over Σ.

Substrings

Let w be a string over the alphabet Σ

- A string z is a ***substring of*** w if z appears consecutively within w;

  Examples: Let $w =$ abracadabra.

  ε, abra, cada, abracadabra, ...

  substrings of w

- A string x is a ***prefix of*** w if x appears at the beginning of w;

  Examples: ε, abra, abracada, ... , abracadabra

  prefixes of w

- A string y is a ***suffix of*** w if y appears at the end of w;

  Examples: ε, abra, cadabra, ... , abracadabra

  suffixes of w

String Concatenation

- Given strings w and w' over the same alphabet, the ***concatenation of w and w'***, written ww', is the string obtained by appending w' to the end of w

  Examples: For $w = \text{ab}$ and $w' = \text{cd}$,

  $ww' = \text{abcd}$, $w'w = \text{cdab}$, $w^3 = \text{ababab}$, ...

$$w^k = \overbrace{ww\cdots w}^{k}$$

Note that...

- The empty string is the *identity element*: $\forall w \,.\, (w\varepsilon = w = \varepsilon w)$;
- If x is a *prefix of w*, then $w = xw'$ for some w';
- If y is a *suffix of w*, then $w = w'y$ for some w';
- If z is a *substring of w*, then $w = w'zw''$ for some w', w'';

last definitions ...

- The ***length of a word*** w, written $|w|$, is the number of symbols it contains.

  Examples: $|\text{abc}| = 3$, $|\text{abracadabra}| = 11$, $|\varepsilon| = 0$.

- The ***reverse of*** w, written w^R, is the string obtained by writing w in the opposite order.

  Examples: $\text{abc}^R = \text{cba}$, $\text{abracadabra}^R = \text{arbadacarba}$,
  $\varepsilon^R = \varepsilon$, $\text{abba}^R = \text{abba}$, $\text{eve}^R = \text{eve}$. palindroms

- If the alphabet Σ is ordered, then the set of strings over Σ can be ordered ***lexicographically***:

$$w \preceq w' \iff (w \text{ prefix of } w') \text{ or } \exists a \leq b \in \Sigma . \begin{cases} ua \text{ prefix of } w \\ ub \text{ prefix of } w' \end{cases}$$

lexicographic order

a simple Model of Computation

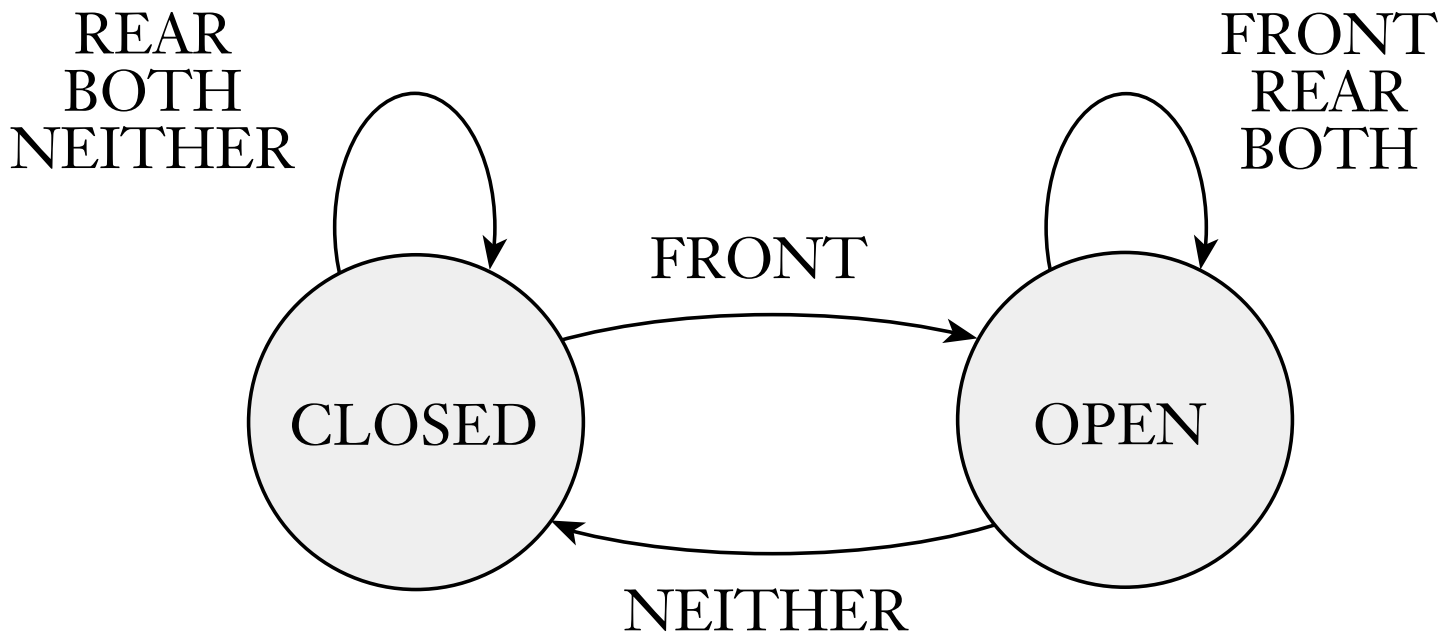
it's an abstract prototype of digital computers used to develop a mathematical theory of computations

"workers in the field, [...] have felt more and more that the notion of Turing machine is too general to serve as an accurate model of actual computers. It is well known that even for simple calculations it is impossible to give an a priori upper bound on the amount of tape a Turing machine will need for any given computation. It is precisely this features that renders Turing's concept unrealistic"

Rabin, M. and Scott, D. *Finite automata and their decision problems*. IBM Journal of Research and Development, vol. 3 (1959), pp. 114 - 125.

Finite automata are models for computers with an extremely limited amount of memory

Controller for automatic door



Two sensors (on front pad; on rear pad), hence 4 input possibilities: BOTH, only FRONT, only REAR, NEITHER

Finite Automaton

Definition

A ***finite automaton*** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite ***set of states***,
- Σ is a finite set called ***alphabet***,
- $\delta\colon Q \times \Sigma \to Q$ is the ***transition function***,
- $q_0 \in Q$ is the ***start state*** (or *initial state*)
- $F \subseteq Q$ is the set of ***accept states*** (or *final states*).

exactly one next state for each input symbol in each state

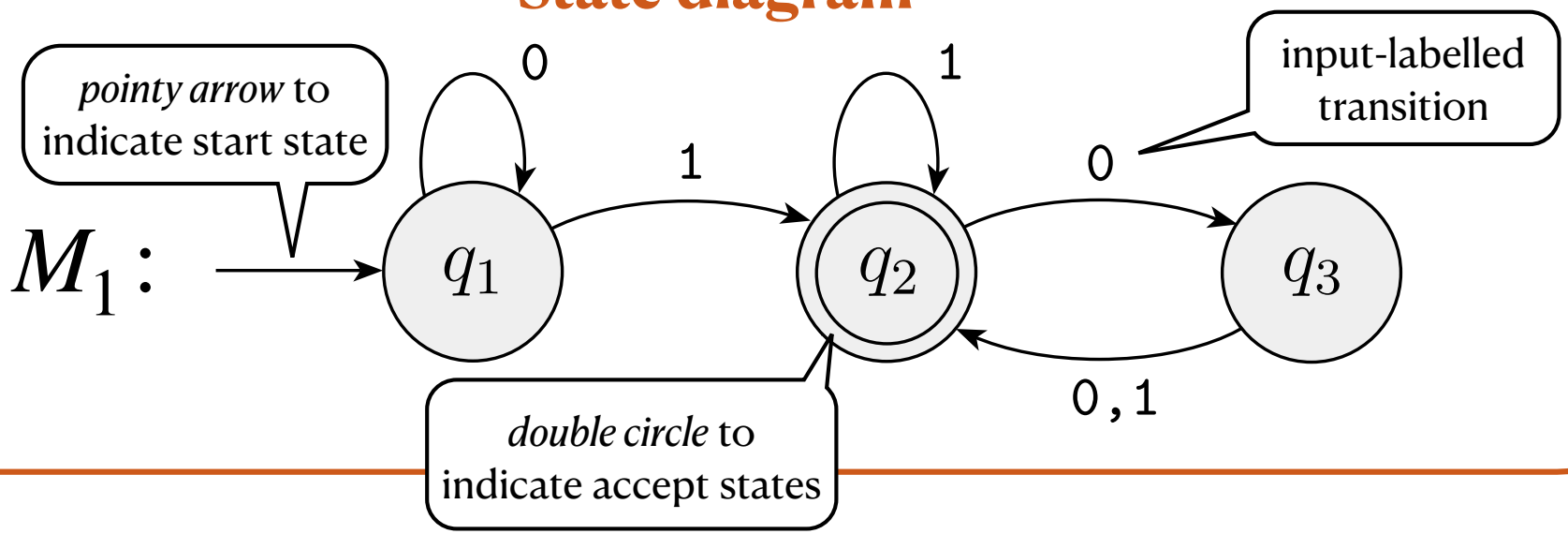
there must always be a starting state

there can be 0 accept states, or several

it is possible that the start state is also an accept state ($q_0 \in F$)

Example of Finite Automaton

State diagram



Formal description

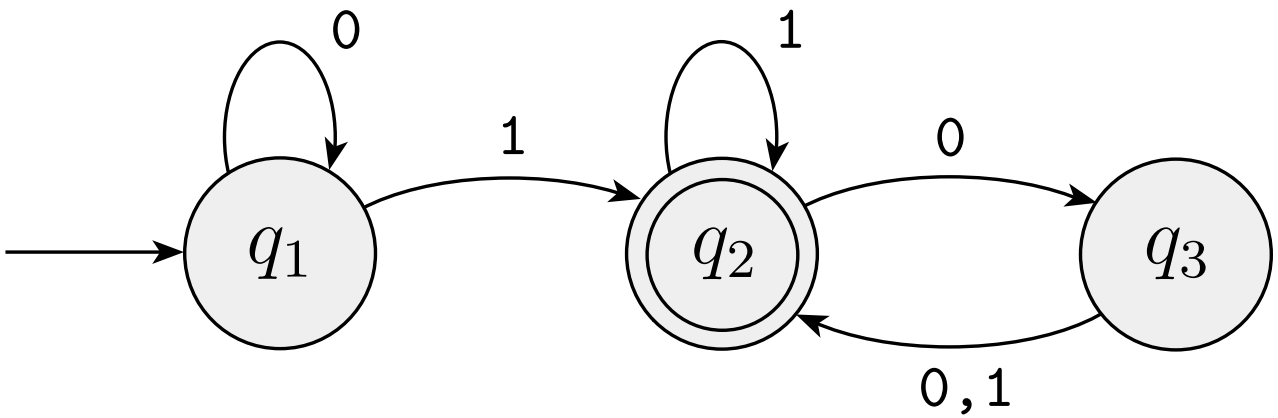
The automaton M_1 is given by the tuple $(Q, \Sigma, \delta, q_0, F)$, where

$$Q = \{q_1, q_2, q_3\} \qquad \Sigma = \{0,1\}$$

$$q_0 = q_1 \qquad F = \{q_2\}$$

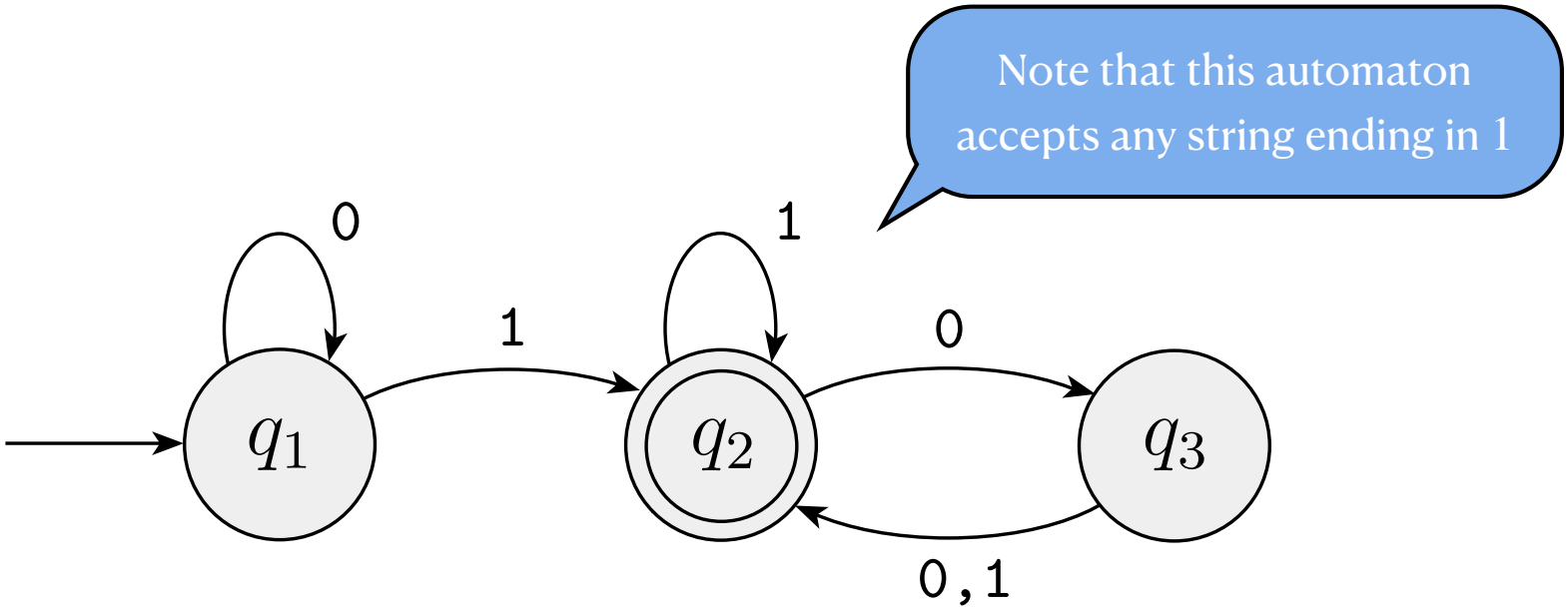
δ	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

$\delta: Q \times \Sigma \to Q$ (tabular definition)



observe how the automaton processes the strings

1 1 0 1	✅ accept
0 0 1 1 0 0	✅ accept
1 1 0 1 0	❌ reject



when an automaton processes a string of symbols over its alphabet, it outputs:

- accept -when it finishes in an accept state
- reject -otherwise

Formal definition of Computation

Definition

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an FA and $w = a_1 a_2 \ldots a_n$ a string where each $a_i \in \Sigma$. Then, ***M accepts w*** if there exists a sequence of states $r_0, r_1, \ldots, r_n$ in Q satisfying the following 3 conditions:

1. $r_0 = q_0$
2. $\delta(r_i, a_{i+1}) = r_{i+1}$, for $i \in \{0, \ldots, n-1\}$
3. $r_n \in F$

$r_0, r_1, \ldots, r_n$ is the ***computation sequence*** for w in M

Definition

- The ***language of M***, denoted by $L(M)$, is $\{w \mid M \text{ accepts } w\}$;
- We say that ***M recognises a language A***, when $A = L(M)$

Example 1

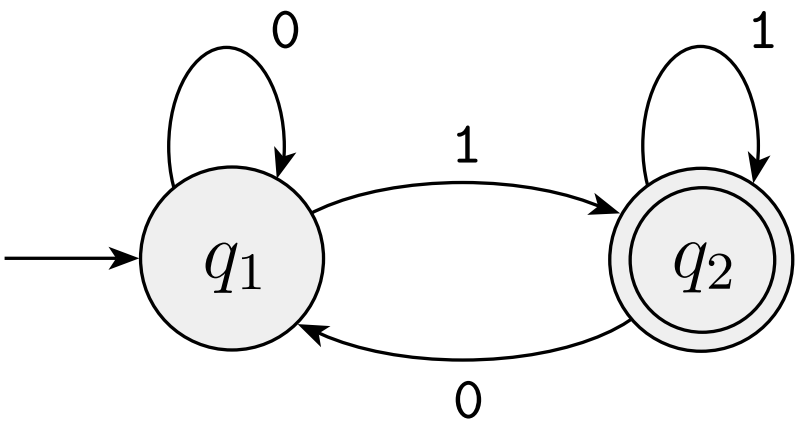
Formal description

$$M_2 = (Q, \Sigma, \delta, q_1, F)$$

where

- $Q = \{q_1, q_2\}$
- $\Sigma = \{0,1\}$
- $F = \{q_2\}$

δ	0	1
q_1	q_1	q_2
q_2	q_1	q_2



Describe the language recognized by M_2

$$L(M_2) = \{w \mid w \text{ ends in } 1\}$$

Example 2

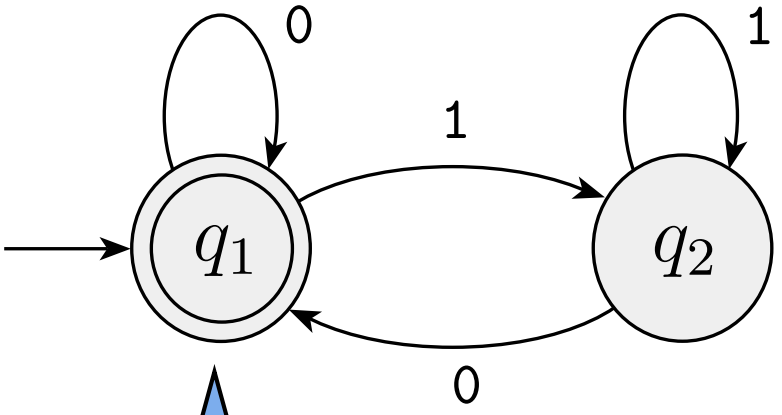
Formal description

$$M_3 = (Q, \Sigma, \delta, q_1, F)$$

where

- $Q = \{q_1, q_2\}$
- $\Sigma = \{0,1\}$
- $F = \{q_1\}$

δ	0	1
q_1	q_1	q_2
q_2	q_1	q_2



whenever the start state of an automaton M is also accepting, then $\varepsilon \in L(M)$

Describe the language recognized by M_3

$$L(M_3) = \{w \mid w \text{ ends in } 0\} \cup \{\varepsilon\}$$

Example 3

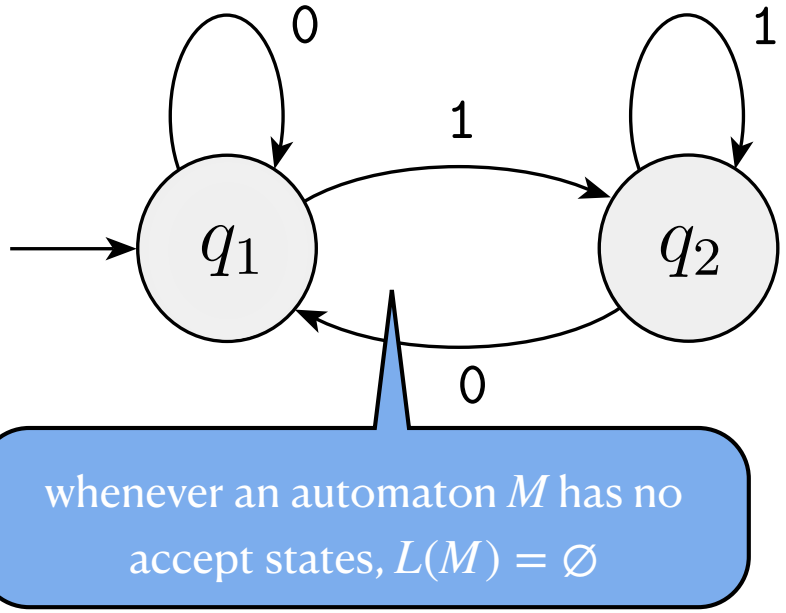
Formal description

$$M_4 = (Q, \Sigma, \delta, q_1, F)$$

where

- $Q = \{q_1, q_2\}$
- $\Sigma = \{0,1\}$
- $F = \varnothing$

δ	0	1
q_1	q_1	q_2
q_2	q_1	q_2

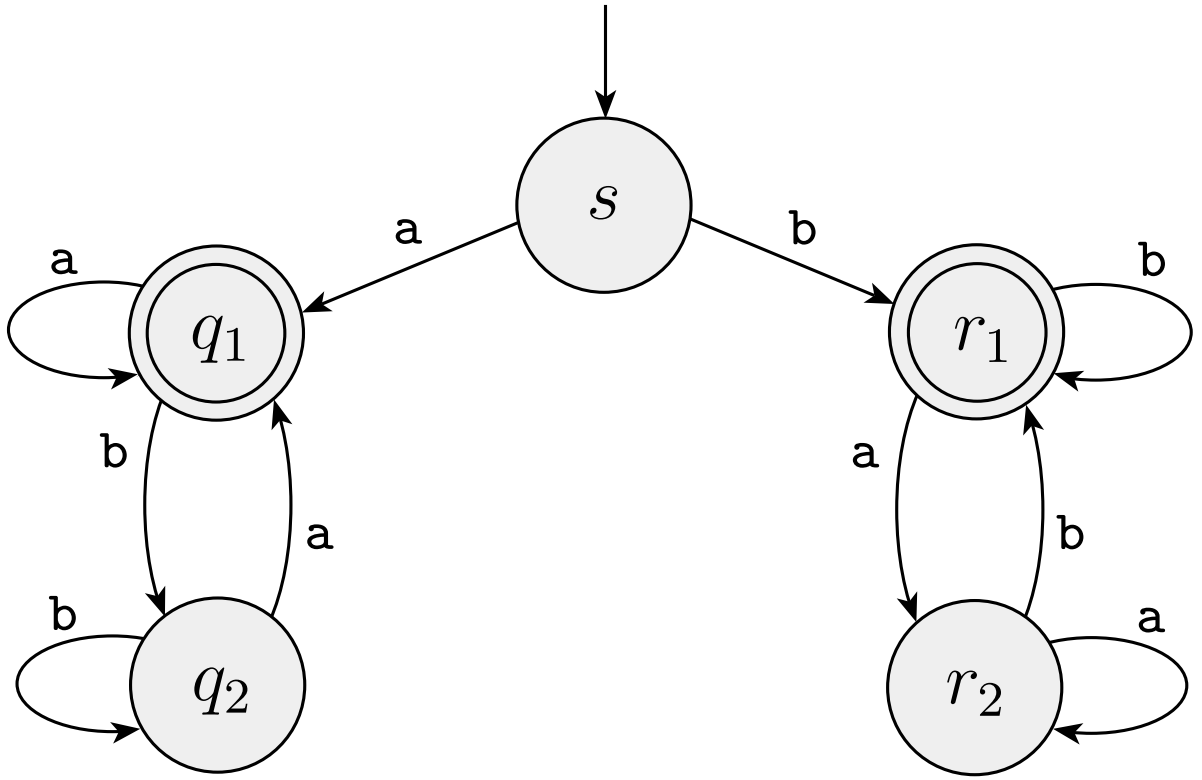


Describe the language recognized by M_4

$$L(M_4) = \varnothing$$

Example 4

M_5:



Describe the language recognized by M_5

$$L(M_5) = \{w \mid w \text{ starts and ends with the same symbol}\}$$
$$= \{xwx \mid x \in \Sigma \text{ and } w \in \Sigma^*\} \cup \{a, b\}$$

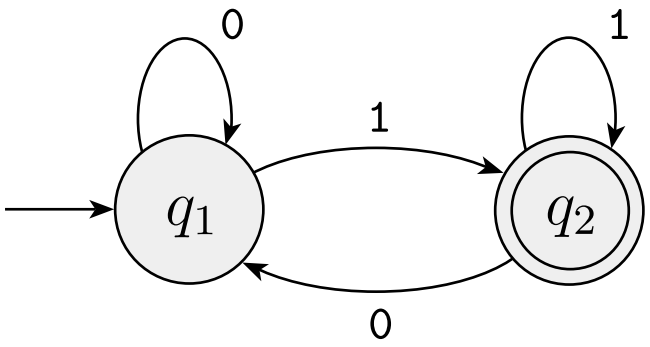
Regular Languages

Definition

A language is ***regular*** is some finite automaton recognizes it.

Example

The language $\text{End}_1 = \{w \mid w \text{ ends in } 1\}$ is regular, as it is recognized by M_2 , in symbols, $L(M_2) = \text{End}_1$.



M_2

Designing Finite Automata

the question of regularity for a language is equivalent
to designing a finite automaton recognizing the given language

A common sense design recipe

The best way to design an automaton is to *put yourself* in the place of the machine and try to processes an input string

- ***Step 1:*** determine the states (they represent what you should remember about the input string read so far)
- ***Step 2:*** determine the transitions (they represent the actions the machine takes when a new symbol is processed)
- ***Step 3:*** determine start and accept states (they are the states where the machine start and completed its task)

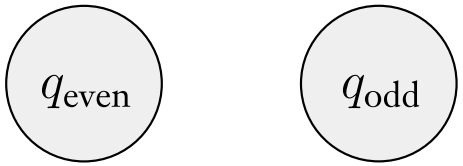
Automata design (Example)

Task

Design an automaton that recognise the language

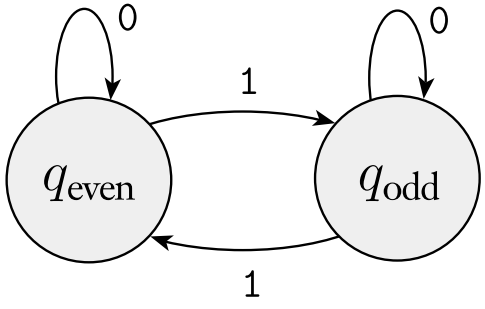
$$\text{Odd} = \{w \in \{0,1\}^* \mid w \text{ has odd number of occurrences of } 1\}$$

Step 1



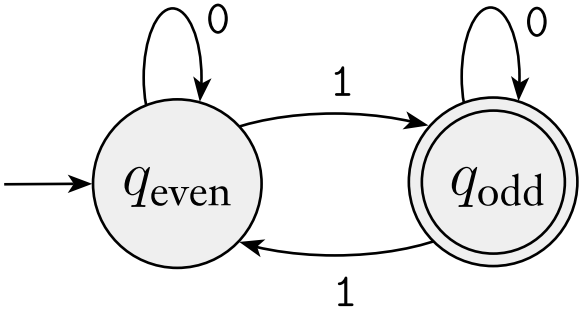
we just need to remember whether the number of inputs equal to 1 processed so far is either *odd* or *even*

Step 2



by reading 1 we switch state;
by reading 0 we stay where we are

Step 3

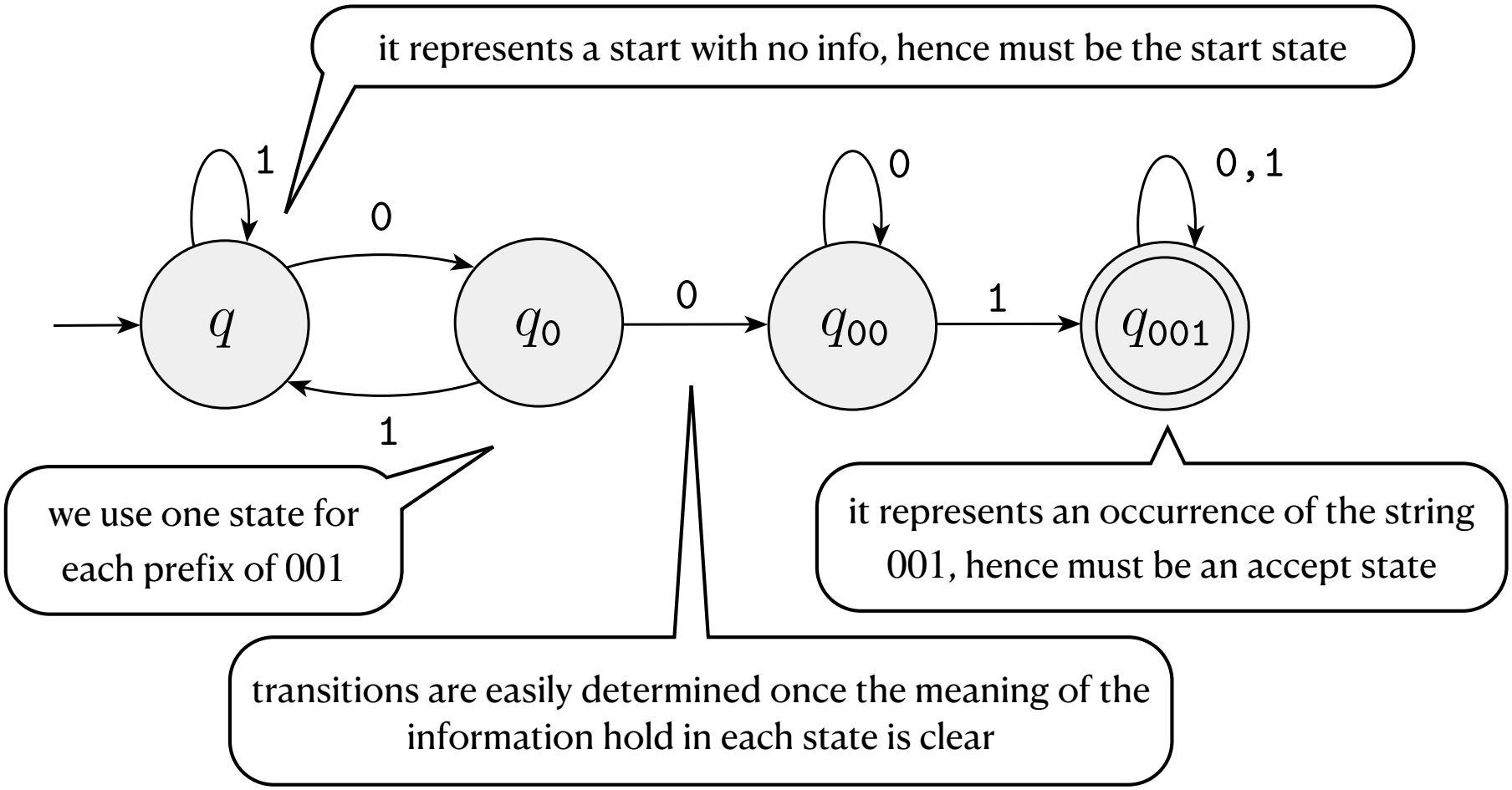


start and accept states are clear

Exercise 1

Show that the following language is regular

$$A_{001} = \{w \in \{0,1\}^* \mid w = x001y \text{ for some } x, y \in \{0,1\}^*\}$$



Exercise 2

Show that the following language is regular

$$A_{111} = \{w \in \{0,1\}^* \mid w = x111y \text{ for some } x, y \in \{0,1\}^*\}$$

same problem for the substring 111

Regular Operations

Definition

Let A and B be two languages. We define the ***regular operations***

- *union*: $A \cup B = \{w \mid w \in A \text{ or } w \in B\}$;
- *concatenation*: $A \circ B = \{ww' \mid w \in A \text{ and } w' \in B\}$;
- *star*: $A^* = \{w_1 w_2 \ldots w_k \mid k \geq 0 \text{ and each } w_i \in A\}$

we consecutively append k strings taken from A
(when $k = 0$, we obtain the empty string ε)

Notable equalities

Let A be any language,

$$A \circ \varnothing = \varnothing = \varnothing \circ A \qquad A \circ \{\varepsilon\} = A = \{\varepsilon\} \circ A$$

Closure Properties

Theorem

The class of regular languages is *closed under union*:

if A and B are regular languages, so is $A \cup B$.

we prove this one

Theorem

The class of regular languages is *closed under intersection*:

if A and B are regular languages, so is $A \cap B$

Theorem

The class of regular languages is *closed under complement*:

if A is a regular language, so is $\overline{A}$.

Theorem

The class of regular languages is *closed under union*:

if A and B are regular languages, so is $A \cup B$.

proof

Assume A, B are regular languages. This means that there exist

$$M_A = (Q_A, \Sigma, q_0^A, \delta_A, F_A) \qquad M_B = (Q_B, \Sigma, q_0^B, \delta_B, F_B)$$

finite automata such that $L(M_A) = A$ and $L(M_B) = B$.

Define the automaton $M = (Q, \Sigma, q_0, \delta, F)$, where

$$Q = Q_A \times Q_B$$

$$q_0 = (q_0^A, q_0^B)$$

$$\delta\colon Q \times \Sigma \to Q\,, \text{ where } \delta((q_1, q_2), a) = (\delta_A(q_1, a), \delta_B(q_2, a))$$

$$F = (F_A \times Q_B) \cup (Q_A \times F_B)$$

continues in the next page...

...continued from the previous page

proof

Next we prove that $L(M) = L(M_A) \cup L(M_B)$.

($\subseteq$) let $w \in L(M)$ and $(q_1, r_1)\ldots(q_n, r_n)$ be the accepting sequence for w in M. By construction, $(q_n, r_n) \in (F_A \times Q_B) \cup (Q_A \times F_B)$. Therefore, $q_n \in F_A$ or $r_n \in F_B$. Moreover, $q_1 = q_0^A$ and $r_1 = q_0^B$. By definition of δ, for each $i \in \{1,\ldots, n-1\}$, we have $\delta_A(q_i, a_{i+1}) = q_{i+1}$ and $\delta_B(r_i, a_{i+1}) = r_{i+1}$. By noting that $q_1, \ldots, q_n$ or $r_1, \ldots, r_n$ is accepting in M_A and M_B, respectively, we conclude that $w \in L(M_A) \cup L(M_B)$.

($\supseteq$) Assume $w \in L(M_A) \cup L(M_B)$. Let $w \in L(M_A)$ [the case $w \in L(M_B)$ is similar] and $q_1, \ldots, q_n$ be the accepting sequence for w in M_A. By construction, the computation for w in M is of the form $(q_1, r_1)\ldots(q_n, r_n)$, for some $r_i \in Q_B$. In particular, since $q_n \in F_A$, we have $(q_n, r_n) \in F$. Consequently, $(q_1, r_1)\ldots(q_n, r_n)$ is accepting in M. Therefore, $w \in L(M)$.

For the above we conclude that $A \cup B = L(M)$. Thus, $A \cup B$ is regular.

more Closure Properties

Theorem

The class of regular languages is *closed under concatenation*:

if A and B are regular languages, so is $A \circ B$.

... the proof of this result is a bit more tricky as for the construction of the automaton recognizing $A \circ B$
we need to determine where to split the input string

to solve this problem in the next lecture
we introduce the concept of *nondeterministic automaton*